



ESCOPO DO PROJETO — Bola AI Express

1. VISÃO GERAL

Componente	Tecnologia	Base de Referência
Portal do Cliente	React + TS + Vite + Tailwind + Firebase	deploy-6934e... (adaptado)
Painel Admin	React + TS + Vite + Tailwind + Firebase	deploy-6934c... (expandido)
WhatsApp Bot	Node.js (Baileys) + Firebase	Novo
Backend/Infra	Firebase (Auth, Firestore, Functions, Hosting)	Existente nos refs

Decisão arquitetural: Manter **Firebase** como backend (Auth, Firestore real-time, Functions para webhooks, Hosting). O React substitui o vanilla JS dos refs — ganha componentização, type-safety, DX melhor.

2. ARQUITETURA DE DADOS (Firestore)

```
collections/
├─ users/                                # Clientes + Admins (role: 'client' | 'admin')
│   └─ {uid}/
│       ├── profile: {nome, tel, endereco, numero, cep, pagamentoPreferido, ...}
│       └─ config: {notificacoes: {whatsapp: true, email: false}}
├─ products/                            # Pods / Cigarros eletrônicos
│   └─ {productId}/
│       ├── nome, descricao, imagem, preco, qtd, categoria, ativo, createdAt
├─ orders/                              # Pedidos unificados (site + WhatsApp + balcão)
│   └─ {orderId}/
│       ├── clienteUid, clienteNome, clienteTel, clienteEndereco
│       ├── itens: [{productId, nome, preco, qtd, subtotal}]
│       ├── total, subtotal, taxaEntrega, desconto
│       ├── formaPagamento: 'pix' | 'dinheiro' | 'cartao'
│       ├── status: 'pendente' | 'confirmado' | 'preparando' | 'saiu_entrega' | 'entregue' |
│       │   'cancelado'
│       ├── origem: 'site' | 'whatsapp' | 'balcao' | 'admin'
│       ├── pix: {txid, qrCode, expiresAt, status: 'pending' | 'paid' | 'expired'}
│       └─ dataIso, dataEntrega, userId (atendente), observacoes
├─ inventory_movements/                 # Movimentações de estoque (auditoria)
│   └─ {movementId}/
│       ├── productId, tipo: 'entrada' | 'saida' | 'ajuste' | 'venda' | 'devolucao'
│       └─ qtd, qtdAnterior, qtdNova, motivo, userId, dataIso
├─ financial/                           # Lançamentos financeiros
│   └─ {lancamentoId}/
│       ├── tipo: 'receita' | 'despesa'
│       └─ categoria: 'venda_produto' | 'entrega' | 'fornecedor' | 'marketing' | 'outros'
```

```

|       |— valor, descricao, formaPagamento, status: 'previsto' | 'realizado' | 'cancelado'
|       |— orderId (se venda), fornecedorId (se despesa), dataIso, userId
|
|— suppliers/                                # Fornecedores
|   |— {supplierId}/
|       |— nome, cnpj, contato, telefone, email, endereco, produtosFornecidos[], observacoes
|
|— whatsapp_sessions/                       # Sessões de conversa ativas
|   |— {phoneNumber}/
|       |— state: 'menu' | 'carrinho' | 'endereco' | 'pagamento' | 'confirmacao' |
'aguardando_humano'
|       |— carrinho: [{productId, qtd}], enderecoTemp, ordemTempId
|       |— ultimaInteracao, clienteUid (se vinculado)
|
|— settings/                                # Configurações globais (doc único)
|   |— general/
|       |— taxaEntrega, valorMinimoEntrega, horarioFuncionamento
|       |— pixConfig: {chave, merchantName, merchantCity}
|       |— whatsappConfig: {numero, token, webhookUrl}
|       |— notificacoes: {novoPedido: true, statusMudanca: true, estoqueBaixo: true}

```

3. PORTAL DO CLIENTE (React + TS)

3.1 Telas & Fluxos

Rota	Tela	Funcionalidades
/	Landing	Hero com logo Bola AI Express, catálogo resumido, CTA "Fazer Pedido"
/login	Auth	Login email/senha + Google + "Esqueci senha" + Link cadastro
/cadastro	Registro	Nome, email, senha, telefone, endereço completo, pagamento preferido
/catalogo	Catálogo	Grid responsivo, filtro por categoria, busca, badge estoque baixo, adicionar ao carrinho
/carrinho	Carrinho (Drawer)	Lista itens, +/- qtd, remove, subtotal, total, botão "Finalizar"
/checkout	Checkout	Confirma endereço, escolhe PIX / Dinheiro na entrega, observações, botão "Confirmar Pedido"
/pix/:orderId	Pagamento PIX	QR Code + Cópia-e-cola + Timer expiração + Polling status + "Já paguei"
/meus-pedidos	Histórico	Lista com status em tempo real (badge colorido), detalhes, comprovante, reordenar
/perfil	Perfil	Edita dados, endereços salvos, preferências, histórico de endereços
/pedido/:id	Detalhe Pedido	Timeline status, itens, total, comprovante PIX, botão "Falar no WhatsApp"

3.2 Features-chave

- **Real-time:** onSnapshot em orders (próprios) + products (disponibilidade)
- **Persistência carrinho:** localStorage + sync ao login
- **PIX:** Firebase Functions gera cobrança (Mercado Pago / Gerencianet / Banco Inter) → salva pix.txid + qrCode no pedido
- **Notificações:** Service Worker (FCM) para push "Pedido confirmado", "Saiu para entrega", "PIX aprovado"
- **PWA:** Manifest + Service Worker (offline-first para catálogo)

4. PAINEL ADMIN (React + TS)

Separado do portal (subdomínio admin.bolaaiexpress.com)

4.1 Layout (Sidebar + Header fixo)

Sidebar:	Header:
└─ Dashboard	└─ Busca global (pedidos, clientes, produtos)
└─ Pedidos	└─ Notificações (badge real-time)
└─ Produtos / Estoque	└─ Perfil admin / Trocar tema / Sair
└─ Clientes	
└─ Financeiro	
└─ Fornecedores	
└─ Relatórios	
└─ WhatsApp Bot	
└─ Configurações	

4.2 Módulos Detalhados

Módulo	Telas / Ações
Dashboard	Cards: Vendas hoje, Pedidos pendentes, Ticket médio, Estoque crítico (<5), Gráfico vendas 7/30 dias, Top 5 produtos, Pedidos recentes (tabela compacta)
Pedidos	Tabela filtrável (status, data, origem, cliente), Ações: Ver detalhes, Alterar status (dropdown), Imprimir comanda, Reimprimir PIX, Marcar pago (manual), Cancelar, Adicionar observação
Produtos/Estoque	CRUD completo, Upload imagem (Firebase Storage), Categorias, Controle de qtd (entrada/saída/ajuste com motivo), Alerta estoque baixo, Importar/Exportar CSV, Histórico movimentações por produto
Clientes	Lista com busca, Ver perfil + histórico pedidos + LTV + ticket médio, Bloquear/Desbloquear, Exportar, Tags (VIP, Atacado, Problema)
Financeiro	Resumo: Receitas/Despesas/Lucro (hoje, semana, mês, custom) · Lançamentos: CRUD receitas/despesas, Vincular a pedido/fornecedor, Conciliação PIX (importar extrato), Contas a pagar/receber, Fluxo de caixa projetado · Relatórios: DRE simplificado, Curva ABC produtos, Vendas por forma pagamento, Por hora/dia
Fornecedores	CRUD, Produtos vinculados, Histórico compras, Contato rápido (WhatsApp/Email)

WhatsApp Bot	Status conexão (QR Code / Conectado / Desconectado), Logs conversas, Templates de mensagem, Regras automação (ex: "Se cliente digita '1' → mostra catálogo"), Forçar transferência para humano
Configurações	Dados da loja, Taxa entrega / Mínimo grátis, Horários, PIX (chave, webhook), WhatsApp (número, token, webhook), Notificações (email, push, WhatsApp), Roles/permisões admin

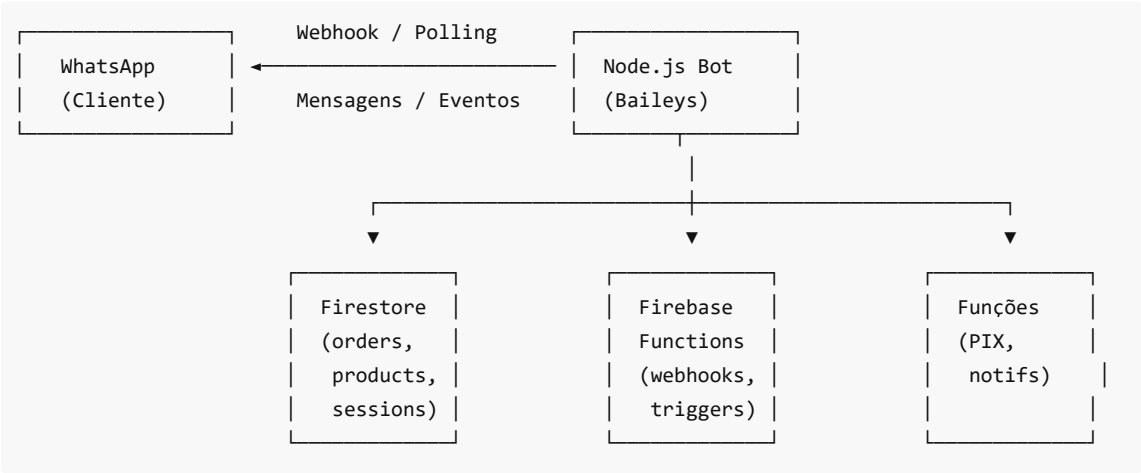
4.3 Permissões (RBAC)

Role	Pedidos	Produtos	Clientes	Financeiro	Fornecedores	Config
Dono	R/W	R/W	R/W	R/W	R/W	R/W
Gerente	R/W	R/W	R	R	R	R
Atendente	R/W*	R	R	-	-	-
Estoque	R	R/W	-	-	R	-

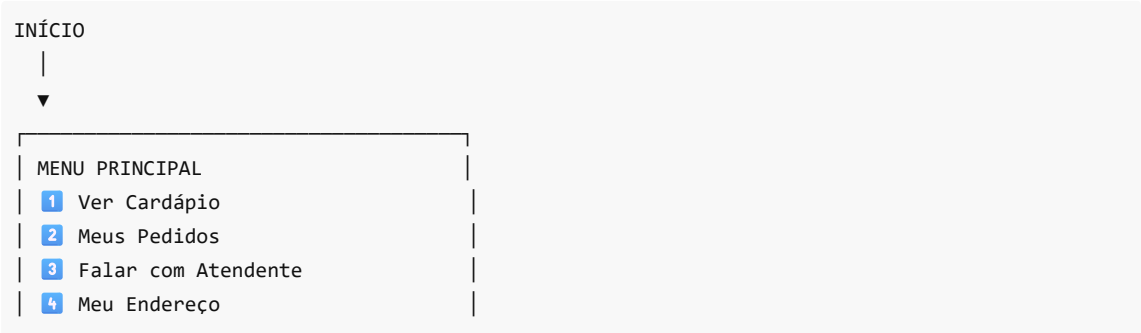
*Atendente só altera status, não apaga.

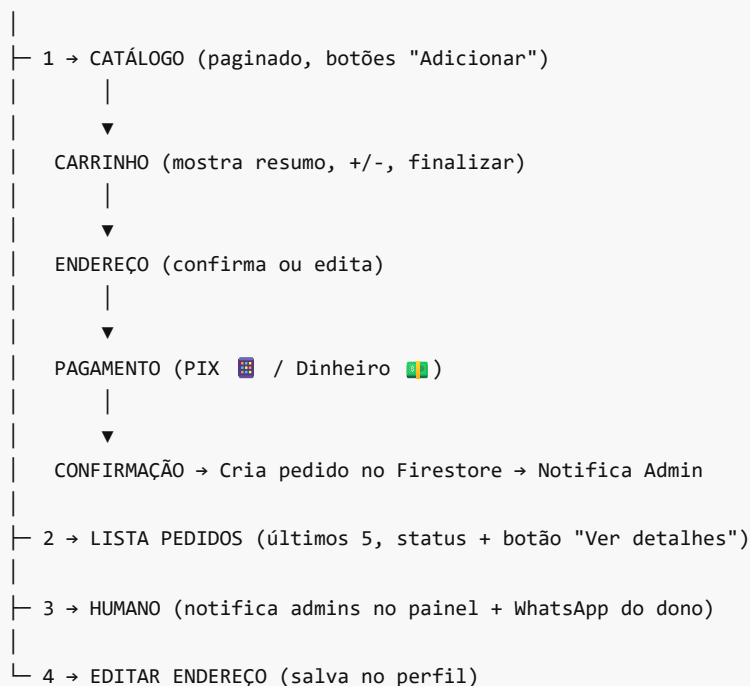
5. WHATSAPP AUTOMATION (Node.js + Baileys)

5.1 Arquitetura



5.2 Fluxo de Conversa (State Machine)





5.3 Comandos Admin (via WhatsApp do dono)

- #pedidos → Lista pedidos pendentes
- #estoque → Produtos com qtd < 5
- #vendas → Resumo dia/semana/mês
- #cliente <tel> → Busca cliente + último pedido
- #status <id> <novo_status> → Altera status pedido

5.4 Integração Bidirecional

Evento Origem	Ação no Firebase	Notificação Destino
Cliente pede no Site	orders.add()	Bot envia msg WhatsApp: "Pedido #123 confirmado!"
Cliente pede no WhatsApp	orders.add(origem: 'whatsapp')	Painel Admin: toast + badge real-time
Admin muda status no Painel	orders.update(status)	Bot envia msg WhatsApp: "Seu pedido #123: SAIU PARA ENTREGA 🚚"
PIX aprovado (webhook)	orders.update(pix.status: 'paid')	Bot: "Pagamento confirmado! 🎉" + Painel: badge verde

6. PAGAMENTO PIX

Provedor	Integração	Por que
Mercado Pago	SDK Node / REST + Webhook	Pix automático, QR Code, split, boleto fallback, docs PT-BR

Gerencianet (Efi)	SDK Node + Webhook	Taxas competitivas, whitelabel, split
Banco Inter / Itaú / Bradesco	API direta	Sem intermediário, menor custo

Recomendação: Mercado Pago — setup rápido, webhook confiável, suporte a `pix` + `cartao futuro`, dashboard próprio.

Fluxo PIX

- 1. Checkout (site/bot) → `POST /api/pix/create` (Firebase Function)
- 2. Function chama Mercado Pago → retorna `txid` , `qr_code` , `qr_code_base64` , `expiration`
- 3. Salva no `orders/{id}.pix` + mostra QR pro cliente
- 4. Cliente paga → Mercado Pago posta webhook → Function valida + `orders.update(pix.status:'paid')` + dispara notificações
- 5. Se expirar (15-30min) → Function cron marca `expired` + libera estoque (se reservado)

7. IDENTIDADE VISUAL — Bola AI Express

Elemento	Especificação
Logo	<code>bolaaia.png</code> (1254x1254) → usar como favicon, apple-touch-icon, header, splash PWA
Cores	Extrair da logo (provavelmente tons de azul/roxo/rosa tech + amarelo/laranja energy) → definir <code>--primary</code> , <code>--secondary</code> , <code>--accent</code> no CSS
Tipografia	Poppins (já usada nos refs) ou Inter / DM Sans — modernas, legíveis
Tema	Dark default (estilo "tech/vape") + Light opcional — CSS Variables como no <code>style.css</code> do ref
Ícones	Lucide React / Phosphor Icons (tree-shakeable, consistentes)
Componentes UI	shadcn/ui (Radix + Tailwind) — acessíveis, customizáveis, copiáveis

8. ESTRUTURA DO MONOREPO (Vite + React + TS)

```
bola-ai-express/
├─ apps/
│   └─ client/                # Portal do Cliente (PWA)
│       └─ src/
│           └─ components/    # UI compartilhada (Button, Card, Input, Modal, Badge, etc.)
│           └─ pages/         # Landing, Auth, Catalogo, Carrinho, Checkout, Pix, Orders,
Perfil
│   └─ hooks/                 # useAuth, useCart, useProducts, useOrders, useTheme
│   └─ context/               # AuthContext, CartContext, ThemeContext
│   └─ services/              # firebase.ts, api.ts (Functions), pix.ts
│   └─ types/                 # index.ts (User, Product, Order, CartItem, etc.)
│   └─ utils/                 # formatters, validators, constants
│   └─ styles/                # globals.css, variables.css
```

```

|   |   |   | index.html
|   |   |   | vite.config.ts
|   |   |   | package.json
|   |   |
|   |   |   | admin/                # Paine! Admin
|   |   |   |   | src/
|   |   |   |   |   | components/    # Sidebar, Header, DataTable, Charts, Forms, Modals
|   |   |   |   |   | pages/        # Dashboard, Orders, Products, Clients, Financial, Suppliers,
|   |   |   |   |   | WhatsApp, Settings
|   |   |   |   |   |   | hooks/      # useAuthAdmin, useRealTime<T>, usePermissions
|   |   |   |   |   |   | context/    # AdminAuthContext, NotificationContext
|   |   |   |   |   |   | services/   # firebase.ts, api.ts, reports.ts, export.ts
|   |   |   |   |   |   | types/      # index.ts (extends client types + Admin-specific)
|   |   |   |   |   |   |   | styles/
|   |   |   |   |   |   | vite.config.ts
|   |   |   |   |   |   | package.json
|   |   |   |   |   |
|   |   |   |   |   | whatsapp-bot/  # Bot WhatsApp (Node.js ESMmodules)
|   |   |   |   |   |   | src/
|   |   |   |   |   |   |   | index.ts    # Entry: conecta Baileys, registra handlers
|   |   |   |   |   |   |   | session/    # Gerencia sessões (Firestore + arquivo local)
|   |   |   |   |   |   |   | state-machine/ # ConversationState, steps, transitions
|   |   |   |   |   |   |   | handlers/    # menu.ts, catalogo.ts, carrinho.ts, checkout.ts, admin.ts
|   |   |   |   |   |   |   | services/    # firebase.ts, pix.ts, notifications.ts
|   |   |   |   |   |   |   | templates/   # Mensagens pré-formatadas (Handlebars)
|   |   |   |   |   |   |   |   | utils/    # formatters, validators, logger
|   |   |   |   |   |   |   | package.json
|   |   |   |   |   |   |   | tsconfig.json
|   |   |   |   |   |
|   |   |   |   | packages/
|   |   |   |   |   | shared-types/      # Types compartilhados (Product, Order, User, etc.)
|   |   |   |   |   | firebase-config/   # Inicialização Firebase (client + admin SDK)
|   |   |   |   |   | ui-components/     # Design System (shadcn-based) – opcional
|   |   |   |   |
|   |   |   |   | functions/             # Firebase Functions (Node 20 + TS)
|   |   |   |   |   | src/
|   |   |   |   |   |   | pix/           # createPix, webhookPix, expirePix
|   |   |   |   |   |   | orders/        # onCreateOrder (notifica bot/admin), onStatusChange
|   |   |   |   |   |   | inventory/     # onStockChange (alerta baixo), reserveStock, releaseStock
|   |   |   |   |   |   | notifications/ # sendPush, sendWhatsApp, sendEmail
|   |   |   |   |   |   | reports/       # dailyReport, weeklyReport (agendados)
|   |   |   |   |   |   |   | utils/     # firebase-admin, mercadopago, bailey-client
|   |   |   |   |   |   | package.json
|   |   |   |   |   |   | tsconfig.json
|   |   |   |   |
|   |   |   |   | firebase.json          # Hosting (client + admin), Functions, Firestore rules,
|   |   |   |   | Storage rules
|   |   |   |   |   | firestore.rules
|   |   |   |   |   | firestore.indexes.json
|   |   |   |   |   | storage.rules
|   |   |   |   |   | package.json       # Root (workspaces: apps/*, packages/*, functions)
|   |   |   |   |   | turbo.json        # Turborepo config (build, dev, lint, typecheck)

```

```
└─ tsconfig.base.json
└─ README.md
```

9. REGRAS FIRESTORE (Segurança)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // USERS - próprio usuário ou admin
    match /users/{uid} {
      allow read, write: if request.auth != null && (request.auth.uid == uid ||
get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin');
      match /{subcollection=**} { allow read, write: if request.auth != null &&
request.auth.uid == uid; }
    }

    // PRODUCTS - leitura pública (ativos), escrita só admin
    match /products/{productId} {
      allow read: if resource.data.ativo == true || request.auth != null;
      allow create, update, delete: if request.auth != null &&
get(/databases/{database}/documents/users/{request.auth.uid}).data.role in ['admin',
'gerente', 'estoque'];
    }

    // ORDERS - cliente vê só seus, admin vê todos
    match /orders/{orderId} {
      allow create: if request.auth != null;
      allow read: if request.auth != null && (resource.data.clienteUid == request.auth.uid
|| get(/databases/{database}/documents/users/{request.auth.uid}).data.role in ['admin',
'gerente', 'atendente']);
      allow update: if request.auth != null && (
        (request.auth.uid == resource.data.clienteUid && request.resource.data.status ==
'cancelado' && resource.data.status == 'pendente') ||
        (get(/databases/{database}/documents/users/{request.auth.uid}).data.role in
['admin', 'gerente', 'atendente'])
      );
    }

    // FINANCIAL - só admin/gerente
    match /financial/{docId} {
      allow read, write: if request.auth != null &&
get(/databases/{database}/documents/users/{request.auth.uid}).data.role in ['admin',
'gerente'];
    }

    // INVENTORY_MOVEMENTS - leitura admin, escrita system/admin
    match /inventory_movements/{docId} {
      allow read: if request.auth != null &&
get(/databases/{database}/documents/users/{request.auth.uid}).data.role in ['admin',
```

```

'gerente', 'estoque'];
  allow create: if request.auth != null;
}

// WHATSAPP_SESSIONS - só bot (service account) + admin
match /whatsapp_sessions/{phone} {
  allow read, write: if request.auth == null || request.auth.token.admin == true;
}

// SETTINGS - leitura pública (configs não sensíveis), escrita admin
match /settings/{docId} {
  allow read: if true;
  allow write: if request.auth != null &&
get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin';
}
}
}

```

10. DEPLOY & CI/CD

Ambiente	Hosting	URL
Produção	Firebase Hosting	https://bolaaexpress.com (cliente) + https://admin.bolaaexpress.com (admin)
Staging	Firebase Hosting (preview channels)	https://bolaaexpress--staging.web.app
Bot/WhatsApp	Railway / Render / Fly.io / VPS (sempre online)	Webhook: https://api.bolaaexpress.com/whatsapp/webhook
Functions	Firebase Functions (Node 20)	Auto-deploy com firebase deploy --only functions

GitHub Actions

```

# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  lint-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4 (node: 20)
      - run: npm ci
      - run: npm run lint && npm run typecheck
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run test

```

```

deploy-preview:
  if: github.event_name == 'pull_request'
  needs: [lint-typecheck, test]
  runs-on: ubuntu-latest
  steps:
    - uses: FirebaseExtended/action-hosting-deploy@v0
      with: {repoToken: ${{ secrets.GITHUB_TOKEN }}, firebaseServiceAccount: ${{
secrets.FIREBASE_SA }}, projectId: bola-ai-express, target: client}
deploy-prod:
  if: github.ref == 'refs/heads/main'
  needs: [lint-typecheck, test]
  runs-on: ubuntu-latest
  steps:
    - run: firebase deploy --project bola-ai-express --only
hosting:client,hosting:admin,functions,firestore,storage

```

11. ROADMAP / FASES

Fase	Entregáveis	Estimativa
0. Setup	Monorepo (Turborepo), Firebase project, CI/CD, Design System base, Types compartilhados	3-5 dias
1. Core Compartilhado	Firebase config (client + admin SDK), Auth (email/Google + roles), Hooks useAuth, useRealTime, Componentes UI base (Button, Input, Card, Table, Modal, Toast, Badge, Avatar, Dropdown, Tabs, Charts)	1-2 semanas
2. Portal Cliente	Landing, Auth, Catálogo, Carrinho, Checkout (PIX + Dinheiro), PIX polling, Meus Pedidos (real-time), Perfil, PWA	2-3 semanas
3. Painel Admin	Layout (Sidebar/Header), Dashboard (charts + cards), Pedidos (CRUD + status + impressão), Produtos/Estoque (CRUD + movimentações), Clientes (CRUD + histórico), Financeiro (lançamentos + relatórios + DRE), Fornecedores, Configurações, RBAC	3-4 semanas
4. WhatsApp Bot	Baileys setup, State machine, Fluxo completo (menu → catálogo → carrinho → endereço → pagamento → confirmação), Admin commands, Webhook bidirecional (site ↔ bot ↔ painel), Sessões persistidas no Firestore	2-3 semanas
5. PIX & Integrações	Firebase Functions (Mercado Pago), Webhooks, Conciliação, Notificações (FCM + WhatsApp + Email), Testes de ponta a ponta	1-2 semanas
6. Polimento & Go-Live	Testes carga, SEO/PWA, Analytics (GA4 + Mixpanel), Logs/Monitoramento (Sentry), Documentação, Treino, Deploy prod	1 semana

Total estimado: 10-15 semanas (1 dev full-stack) — pode paralelizar: Fase 2 + 3 simultâneas se 2 devs.

12. DECISÕES TÉCNICAS PENDENTES

Decisão	Opções	Minha Recomendação
---------	--------	--------------------

Provedor PIX	Mercado Pago / Gerencianet / Banco Inter / Asaas	Mercado Pago (setup rápido, boas taxas, dashboard)
Hospedagem Bot	Railway (\$5/mês) / Render / Fly.io / VPS próprio (DigitalOcean/Hetzner)	Railway (simples, logs, auto-deploy, Redis incluso)
Charts Admin	Recharts / Chart.js / Tremor / Nivo	Recharts (React-native, leve, bons exemplos)
Tabelas Admin	TanStack Table (headless) / AG Grid (Community) / MUI	TanStack Table (grátis, flexível, TS-first)
Formulários	React Hook Form + Zod / TanStack Form	RHF + Zod (padrão mercado, validação schema)
Notificações Push	FCM (Firebase) / OneSignal	FCM (já no Firebase, grátis, ilimitado)
Testes E2E	Playwright / Cypress	Playwright (multi-browser, rápido, API testing)
Logs/Erros	Sentry / LogRocket / Datadog	Sentry (generoso free tier, integra React + Node)

13. PRÓXIMOS PASSOS IMEDIATOS

1. **Confirmar o escopo acima** (Ajustes, remoções, prioridades)
2. **Definir as decisões pendentes** (Tabela 12)
3. **Inicializar o monorepo** com Turborepo + Vite + TS + Tailwind + shadcn/ui + Firebase
4. **Criar os types compartilhados** baseados no modelo de dados (Seção 2)
5. **Configurar Firebase project** (Auth, Firestore, Functions, Hosting, Storage)
6. **Começar Fase 1** (Core compartilhado + Design System)